Aero Hand Open ROS 2 on WSL

Environment setup, USB attachment, Python dependencies, camera fixes, and launch commands

Prepared for the Windows + Ubuntu 22.04 WSL workflow. Last updated: 2026-06-19.

This guide compiles the troubleshooting steps from the session into a clean checklist for launching the webcam teleoperation file from WSL or from a Windows launcher.

Assumptions used below:

- Windows user: djsz9
- Ubuntu/WSL user: will
- WSL distro: Ubuntu-22.04
- ROS distribution: Humble
- Workspace: /home/will/chestnut/aero-hand-open/ros2
- Launch file: src/launch_files/webcam_teleop_launch/webcam_teleop.launch.py

Quick table of contents

1. One-time Windows setup
2. Ubuntu/ROS setup
3. Python dependency fixes
4. Build and source the workspace
5. Attach hands and camera to WSL
6. Hand port mapping and launch commands
7. Camera preview and green-screen fixes
8. Serial stability and false-reading reduction
9. Windows launcher scripts

1. One-time Windows setup

Install WSL/Ubuntu 22.04 and usbipd-win so Windows USB devices can be attached into WSL 2.

Install Ubuntu 22.04 from PowerShell if the Microsoft Store listing is hard to find:

```
wsl --install -d Ubuntu-22.04
```

Install or update usbipd-win in Windows PowerShell:

```
winget install --interactive --exact dorssel.usbipd-win
```

> After installing usbipd-win, close and reopen PowerShell. Run PowerShell as Administrator for binding devices.

Check usbipd is installed and list USB devices:

```
usbipd list
```

If PowerShell says usbipd is not recognized, try reopening PowerShell or running the executable directly:

```
"C:\Program Files\usbipd-win\usbipd.exe" list
```

Update/restart WSL when USB or GUI behavior is odd:

```
wsl --update
wsl --shutdown
```

2. Ubuntu/ROS setup

Inside Ubuntu/WSL, install system packages needed for building, Python installs, camera diagnostics, and the ncurses header needed by manus_ros2.

Base packages:

```
sudo apt update
sudo apt install -y python3-pip python3-venv v4l-utils libncurses-dev
```

Optional tools for GUI/camera testing:

```
sudo apt install -y x11-apps ffmpeg guvcview
```

Source ROS Humble:

```
source /opt/ros/humble/setup.bash
```

Use rosdep to install missing system dependencies from the workspace:

```
cd /home/will/chestnut/aero-hand-open/ros2
sudo rosdep init 2>/dev/null || true
rosdep update
rosdep install --from-paths src --ignore-src -r -y
```

> Do not use sudo colcon build. It can make build/, install/, and log/ owned by root, which later causes permission errors.

3. Python dependency fixes

Install the project requirements, then pin MediaPipe and NumPy to versions that work with the legacy mp.solutions API and dex-retargeting.

Install requirements:

```
cd /home/will/chestnut/aero-hand-open/ros2
python3 -m pip install --upgrade pip
python3 -m pip install -r requirements.txt
```

If pip warns that scripts such as torchrun are installed in ~/.local/bin and that folder is not on PATH, this is usually harmless for ROS imports. To fix it anyway:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Fix MediaPipe for code that calls mp.solutions.hands.Hands:

```
python3 -m pip uninstall -y mediapipe
python3 -m pip install --user --force-reinstall "mediapipe==0.10.14"
```

Restore NumPy compatibility with dex-retargeting:

```
python3 -m pip install --user --force-reinstall "numpy==1.26.4"
```

Test imports:

```
python3 -c "import mediapipe as mp; print(mp.__version__); print(mp.solutions.hands.Hands)"
python3 -c "import numpy; print('numpy', numpy.__version__)"
python3 -c "import dex_retargeting; print('dex_retargeting ok')"
python3 -c "from aero_open_sdk.aero_hand import AeroHand; print('sdk ok')"
```

> A Matplotlib warning about Axes3D is not important for this launch unless you need 3D plotting.

4. Build and source the workspace

Clean old build products if needed and build:

```
cd /home/will/chestnut/aero-hand-open/ros2
rm -rf build install log
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Verify packages are visible:

```
ros2 pkg list | grep webcam_mocap
ros2 pkg list | grep aero_hand_open
```

> If ROS only searches /opt/ros/humble, the overlay is not sourced. Run source install/setup.bash from the workspace root.

5. Attach hands and camera to WSL

Your devices identified during troubleshooting:

Device	Windows BUSID	USB ID	Expected Linux path
Newer hand	1-10	1a86:7523 CH340	/dev/serial/by-id/usb-1a86_USB_Serial-if00-port0
Older hand	1-11	303a:1001 Espressif	/dev/serial/by-id/usb-Espressif_USB_JTAG_serial_debug_unit_1C:DB:D4:5C:9E:F8-if00
Camera	Use usbipd list	USB webcam	/dev/video0, sometimes /dev/video1

Bind once in Windows PowerShell as Administrator:

```
usbipd bind --busid 1-10
usbipd bind --busid 1-11
usbipd bind --busid <CAMERA_BUSID>
```

Attach each time you start the WSL session or reconnect devices:

```
usbipd attach --wsl --busid 1-10
usbipd attach --wsl --busid 1-11
usbipd attach --wsl --busid <CAMERA_BUSID>
```

Verify inside Ubuntu:

```
lsusb
ls -l /dev/serial/by-id/
ls /dev/ttyUSB* 2>/dev/null
ls /dev/ttyACM* 2>/dev/null
ls -l /dev/video* 2>/dev/null
```

If the CH340 hand does not appear as /dev/ttyUSB*, load the driver:

```
sudo modprobe ch341
ls /dev/ttyUSB*
ls -l /dev/serial/by-id/
```

> If a device says Shared in usbipd list, it is available for WSL but may not be attached yet. Use usbipd attach --wsl --busid ...

6. Permissions and exact launch commands

Add your Ubuntu user to the serial and video groups. You must restart WSL afterward for group changes to apply.

Add groups:

```
sudo usermod -aG dialout $USER
sudo usermod -aG video $USER
```

Restart WSL from Windows PowerShell, then reopen Ubuntu:

```
wsl --shutdown
```

Check groups and optionally apply temporary permissions after USB attach:

```
groups
sudo chmod a+rw /dev/ttyUSB* 2>/dev/null || true
sudo chmod a+rw /dev/ttyACM* 2>/dev/null || true
sudo chmod a+rw /dev/video* 2>/dev/null || true
```

> For two hands, avoid auto-detection. Pass the exact by-id ports so the newer CH340 hand is not skipped.

Launch with newer hand on right and older hand on left:

```
cd /home/will/chestnut/aero-hand-open/ros2
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch src/launch_files/webcam_teleop_launch/webcam_teleop.launch.py \
right_hand_port:=/dev/serial/by-id/usb-1a86_USB_Serial-if00-port0 \
left_hand_port:=/dev/serial/by-id/usb-
Espressif_USB_JTAG_serial_debug_unit_1C:DB:D4:5C:9E:F8-if00 \
feedback_frequency:=10.0
```

If the physical hands are reversed, swap ports:

```
ros2 launch src/launch_files/webcam_teleop_launch/webcam_teleop.launch.py \
right_hand_port:=/dev/serial/by-id/usb-
Espressif_USB_JTAG_serial_debug_unit_1C:DB:D4:5C:9E:F8-if00 \
left_hand_port:=/dev/serial/by-id/usb-1a86_USB_Serial-if00-port0 \
feedback_frequency:=10.0
```

7. Camera preview and green-screen fixes

The camera appeared to open but showed a green preview. The v4l2 output showed /dev/video0 supports YUYV and MJPG, including 640x480 at 30 fps. Force a known-good format and use the V4L2 backend.

Force camera format before launch:

```
v4l2-ctl -d /dev/video0 \
--set-fmt-video=width=640,height=480,pixelformat=MJPG \
--set-parm=30
```

Test camera outside ROS:

```
python3 - <<'PY'
import cv2
cap = cv2.VideoCapture(0, cv2.CAP_V4L2)
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
cap.set(cv2.CAP_PROP_FPS, 30)
print('opened:', cap.isOpened())
ok, frame = cap.read()
print('read:', ok, 'shape:', None if frame is None else frame.shape)
if ok:
cv2.imwrite('camera_test.jpg', frame)
print('saved camera_test.jpg')
cap.release()
PY
```

Open the generated image folder from Windows:

```
explorer.exe .
```

If MJPG is still green, try YUYV by changing both the v4l2-ctl pixelformat and the OpenCV FOURCC.

```
v4l2-ctl -d /dev/video0 --set-fmt-video=width=640,height=480,pixelformat=YUYV --set-parm=30
```

Patch webcam_mocap.py so it opens the camera like this:

```
## Video Capture
self.cap = cv2.VideoCapture(camera_idx, cv2.CAP_V4L2)
self.cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*"MJPG"))
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
self.cap.set(cv2.CAP_PROP_FPS, fps)
self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

if not self.cap.isOpened():
self.get_logger().error(f"Could not open video device at {camera_idx}.")
return
```

Also check ret before flipping the frame:

```
ret, frame_bgr = self.cap.read()
if not ret or frame_bgr is None:
self.get_logger().error("Could not read frame from video device.")
return

frame_bgr = cv2.flip(frame_bgr, 1)
```

If the standalone OpenCV test is green too, the problem is likely WSL camera passthrough. Try a different webcam/USB port, detach and reattach with usbipd, or run the camera node on native Ubuntu.

8. Serial stability and false-reading reduction

Warnings like 'Expected 34, got 37', timeouts, and write timeouts usually mean the serial stream is overloaded or desynchronized. The biggest improvement was lowering feedback_frequency from 100 Hz to 10-20 Hz.

Use a lower feedback frequency:

```
ros2 launch src/launch_files/webcam_teleop_launch/webcam_teleop.launch.py \
right_hand_port:=/dev/serial/by-id/usb-1a86_USB_Serial-if00-port0 \
left_hand_port:=/dev/serial/by-id/usb-
Espressif_USB_JTAG_serial_debug_unit_1C:DB:D4:5C:9E:F8-if00 \
feedback_frequency:=10.0
```

Optional SDK timeout patch:

Find the SDK file:

```
python3 -c "import aero_open_sdk.aero_hand as ah; print(ah.__file__)"
```

Change this line:

```
self.ser = Serial(port, baudrate, timeout=0.01, write_timeout=0.01)
```

to:

```
self.ser = Serial(port, baudrate, timeout=0.05, write_timeout=0.05)
self.ser.inter_byte_timeout = 0.005
```

If editing the local SDK, reinstall editable:

```
cd /home/will/chestnut/aero-hand-open/sdk
python3 -m pip install -e .
```

Optional ROS node safety check: only publish state arrays when they are valid float lists of the expected length. This prevents bad serial reads from becoming bogus ROS message values.

```
def _valid_float_list(self, values, name, expected_len=7):
if values is None:
raise ValueError(f"{name} returned None")
if len(values) != expected_len:
raise ValueError(f"{name} expected {expected_len} values, got {len(values)}")
values = [float(v) for v in values]
if not np.all(np.isfinite(values)):
raise ValueError(f"{name} contained NaN or Inf")
return values
```

9. Windows launcher scripts

Use a simple ASCII path such as C:\aero_launcher instead of OneDrive Desktop paths with non-ASCII characters. This avoids batch-file encoding issues.

Create a launcher folder:

```
mkdir C:\aero_launcher
```

Create C:\aero_launcher\launch_aero_hand.ps1 with this content. Replace the camera BUSID.

```
$Distro = "Ubuntu-22.04"

# USB bus IDs from usbipd list
$NewHandBusId = "1-10" # CH340 newer hand
$OldHandBusId = "1-11" # Espressif older hand
$CameraBusId = "PUT_CAMERA_BUSID_HERE"

Write-Host "Attaching newer CH340 hand..."
usbipd attach --wsl --busid $NewHandBusId

Write-Host "Attaching older Espressif hand..."
usbipd attach --wsl --busid $OldHandBusId

Write-Host "Attaching camera..."
usbipd attach --wsl --busid $CameraBusId

Start-Sleep -Seconds 2

$LinuxCommand = @'
cd /home/will/chestnut/aero-hand-open/ros2
source /opt/ros/humble/setup.bash
source install/setup.bash

sudo chmod a+rw /dev/ttyUSB* 2>/dev/null || true
sudo chmod a+rw /dev/ttyACM* 2>/dev/null || true
sudo chmod a+rw /dev/video* 2>/dev/null || true

if command -v v4l2-ctl >/dev/null 2>&1 && [ -e /dev/video0 ]; then
v4l2-ctl -d /dev/video0 --set-fmt-video=width=640,height=480,pixelformat=MJPG --set-
parm=30 || true
fi

ros2 launch src/launch_files/webcam_teleop_launch/webcam_teleop.launch.py \
right_hand_port:=/dev/serial/by-id/usb-1a86_USB_Serial-if00-port0 \
left_hand_port:=/dev/serial/by-id/usb-
Espressif_USB_JTAG_serial_debug_unit_1C:DB:D4:5C:9E:F8-if00 \
feedback_frequency:=10.0
'@

# Remove Windows carriage returns before sending to bash
$LinuxCommand = $LinuxCommand -replace "`r", ""

wsl -d $Distro -- bash -lc $LinuxCommand
```

Create C:\aero_launcher\launch_aero_hand.bat:

```
@echo off
powershell -ExecutionPolicy Bypass -File "C:\aero_launcher\launch_aero_hand.ps1"
pause
```

If you need the camera preview window and it does not show from the .bat file, use the .bat only to attach USB devices, then open an Ubuntu/Windows Terminal session and run the ROS launch visibly. GUI preview windows can behave differently when launched from a non-interactive batch process.

GUI preview test

```
wsl -d Ubuntu-22.04 -- bash -lc "echo DISPLAY=$DISPLAY; echo
WAYLAND_DISPLAY=$WAYLAND_DISPLAY; python3 - <<'PY'
import cv2, numpy as np
img = np.zeros((300,500,3), dtype=np.uint8)
cv2.putText(img, 'WSL GUI TEST', (40,160), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255,255,255), 2)
cv2.imshow('test window', img)
cv2.waitKey(3000)
cv2.destroyAllWindows()
PY"
```

Quick troubleshooting checklist

Symptom	Likely cause	Try this
package webcam_mocap not found	Workspace overlay not sourced	source /opt/ros/humble/setup.bash; source install/setup.bash
ncursesw/ncurses.h missing	Missing ncurses dev headers	sudo apt install libncurses-dev
No module named mediapipe	Python deps missing	python3 -m pip install -r requirements.txt
mediapipe has no attribute solutions	Too-new MediaPipe API	pip install --user --force-reinstall mediapipe==0.10.14
Permission denied on /dev/serial/by-id	User not in dialout or no chmod	sudo usermod -aG dialout $USER; wsl --shutdown; chmod a+rw tty
Permission denied on /dev/video0	User not in video or no chmod	sudo usermod -aG video $USER; wsl --shutdown; chmod a+rw /dev/video*
Expected 34 got 37 / timeouts	Serial overload/desync	Use feedback_frequency:=10.0 and consider longer SDK timeouts
Green camera preview	WSL/OpenCV pixel format issue	Force CAP_V4L2 + MJPG 640x480, or try YUYV
.bat cannot find path with strange characters	Encoding issue in Desktop/OneDrive path	Move scripts to C:\aero_launcher

Recommended normal startup order

bullet Open Windows PowerShell and attach the newer hand, older hand, and camera with usbipd attach --wsl.

bullet Open Ubuntu/WSL or Windows Terminal Ubuntu tab.

bullet Confirm /dev/serial/by-id and /dev/video0 exist.

bullet Source ROS and the workspace overlay.

bullet Launch with exact hand ports and feedback_frequency:=10.0.

Final launch command:

```
cd /home/will/chestnut/aero-hand-open/ros2
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch src/launch_files/webcam_teleop_launch/webcam_teleop.launch.py \
right_hand_port:=/dev/serial/by-id/usb-1a86_USB_Serial-if00-port0 \
left_hand_port:=/dev/serial/by-id/usb-
Espressif_USB_JTAG_serial_debug_unit_1C:DB:D4:5C:9E:F8-if00 \
feedback_frequency:=10.0
```